# minicut 動画編集エディタ

組み込み LuaJIT & プラグイン API リファレンスマニュアル

LuaJIT 5.1 ランタイム ドッキング可能ツールパネル ツールシェルフドック ネットワークライブラリ

(libcurl) FreeType テキスト生成

## 1. はじめに & アーキテクチャ

**minicut** は高速な **LuaJIT** スクリプトエンジンを内蔵しています。 タイムライン編集、クリップ配置、キーフレームアニメーション、自由な UI ツールパネルの構築を自動化できます。

### スクリプト配置ディレクトリ

| 配置場所 | 起動契機 / 役割 | 説明 |
| --- | --- | --- |
| `~/.config/minicut/autorun/` | 起動時自動実行 | 起動時に自動読み込みされ、メニューバー下の「ドック」にアイコンが登録されます。 |
| `~/.config/minicut/scripts/` | メニュークリック時 | メニューバーの「スクリプト」メニューに表示され、クリック時に実行されます。 |
| `~/.config/minicut/icons/` | ドックアイコン画像 | スクリプト名と一致する PNG アイコン画像（例: `keyframe_animator.png`）。 |

## 2. プラグインスクリプト構成

```
-- ========================================================================

-- スクリプト例: my_tool.lua

-- ========================================================================



local target_track = 0

local clip_scale = 1.0



function execute_action()

    minicut.push_undo()

    local clip_id = timeline.place_clip("sample.png", target_track,
timeline.get_playhead(), 4.0)

    if clip_id and clip_id ~= "" then

        clip.set_scale(clip_id, clip_scale)

        timeline.add_keyframe(clip_id, "opacity", 0.0, 0.0)

        timeline.add_keyframe(clip_id, "opacity", 1.0, 1.0)

        minicut.set_status("クリップを配置しました: " .. clip_id)

    end

end



function choose_file()

    gui.file_dialog("ファイルを選択", function(path)

        if path and path ~= "" then minicut.set_status("選択: " .. path) end

    end)
```

```
end



function plugin:init_panel()

    gui.text("メディア配置ツール")

    gui.separator()

    target_track = gui.slider_int("トラック番号", target_track, 0, 4)

    clip_scale = gui.slider_float("拡大率", clip_scale, 0.1, 4.0)

    gui.button("参照...", choose_file)

    gui.button("再生ヘッド位置に配置", execute_action)

end
```

## 3. コアモジュール API

### 3.1. プロジェクト操作 (`project` / グローバル)

| 関数 | 戻り値 | 説明 |
| --- | --- | --- |
| `project_save([path])` | `boolean, string` | プロジェクトを `.minicut` ファイルに保存します。 |
| `project_load(path)` | `boolean, string` | 指定パスからプロジェクトを読み込みます。 |
| `project.get_name()` | `string` | プロジェクト名を取得します。 |
| `project.set_name(name)` | `void` | プロジェクト名を設定します。 |

### 3.2. ネットワーク通信 (`net`)

| 関数 | 戻り値 | 説明 |
| --- | --- | --- |
| `net.get(url)` | `table` | HTTP GET リクエストを実行します。 |
| `net.post(url, body)` | `table` | HTTP POST リクエストを送信します。 |
| `net.download(url, dest)` | `boolean` | リモートファイルをローカルへ直接ダウンロードします。 |

### 3.3. タイムライン & トラック (`timeline`)

| 関数 | 戻り値 | 説明 |
| --- | --- | --- |
| `timeline.get_playhead()` | `number` | 再生ヘッドの現在時刻（秒）を取得します。 |
| `timeline.set_playhead(t)` | `void` | 再生ヘッドを時刻 `t` 秒へ移動します。 |
| `timeline.place_clip(path, trk, t, dur)` | `string` | 動画/音声/画像をトラックに配置します。 |
| `timeline.add_keyframe(id, prop, t, val)` | `void` | アニメーションキーフレームを追加します。 |
| `timeline.clear_keyframes(id, prop)` | `void` | 特定プロパティの全キーフレームを消去します。 |
| `timeline.move_track_up(idx)` | `void` | トラックを上へ移動します。 |
| `timeline.move_track_down(idx)` | `void` | トラックを下へ移動します。 |
| `timeline.set_track_name(idx, name)` | `void` | トラック名を変更します。 |

### 3.4. クリップ変形 (`clip`)

| 関数 | 戻り値 | 説明 |
| --- | --- | --- |
| `clip.get_position(id)` | `number,` | 中心基準の (x, y) 座標を取得します。 |

| 関数 | 戻り値 | 説明 |
| --- | --- | --- |
| | `number` | |
| `clip.set_position(id, x, y)` | `void` | 中心座標を設定します。 |
| `clip.get_scale(id)` | `number` | スケール倍率（1.0 = 100%）を取得します。 |
| `clip.set_scale(id, s)` | `void` | スケール倍率を設定します。 |
| `clip.get_opacity(id)` | `number` | 不透明度 [0.0, 1.0] を取得します。 |
| `clip.set_opacity(id, o)` | `void` | 不透明度を設定します。 |
| `clip.get_volume(id)` | `number` | 音声ボリュームゲイン [0.0, 2.0] を取得します。 |
| `clip.set_volume(id, v)` | `void` | 音声ボリュームゲインを設定します。 |
| `clip.get_transform(clip_id)` | `table` | 変形情報テーブル `{x, y, scale, rotation}` を取得します。 |
| `clip.set_transform(clip_id, x, y, s, r)` | `void` | 位置(X, Y)、拡大率、回転角を一括設定します。 |
| `clip.reset_transform(clip_id)` | `void` | 変形を初期状態（x=0, y=0, scale=1.0, rotation=0）にリセットします。 |

### 3.5. GUI ウィジェット (`gui`)

| 関数 | 戻り値 | 説明 |
| --- | --- | --- |
| `gui.text(text)` | `void` | テキストを表示します。 |
| `gui.text_colored(r, g, b, a, text)` | `void` | 色付きテキストを表示します。 |
| `gui.button(label, [callback])` | `boolean` | ボタンを表示しクリック時に実行します。 |

| 関数 | 戻り値 | 説明 |
| --- | --- | --- |
| `gui.slider_float(label, val, min, max)` | `number` | 浮動小数点スライダーを表示します。 |
| `gui.slider_int(label, val, min, max)` | `integer` | 整数スライダーを表示します。 |
| `gui.input_text(label, text)` | `string` | テキスト入力フィールドを表示します。 |
| `gui.file_dialog(title, cb, [filter])` | `void` | ファイルダイアログを開き結果を返します。 |
| `gui.popout_preview(popped)` | `void` | プレビュー画面を独立ウィンドウに分離（true）またはドッキング（false）します。 |
| `gui.is_preview_popped_out()` | `boolean` | プレビューが現在分離されているかを取得します。 |
| `gui.set_preview_aspect_mode(mode)` | `void` | アスペクト比モード（0:自動, 1:16:9, 2:9:16, 3:4:3, 4:1:1, 5:21:9, 6:4:5）を設定します。 |
| `gui.get_preview_aspect_mode()` | `integer` | 現在のアスペクト比モード（0〜6）を取得します。 |
| `gui.set_transform_tool(tool)` | `void` | ギズモ変形ツール（"select", "translate", "scale", "rotate"）を設定します。 |
| `gui.get_transform_tool()` | `string` | 現在の変形ツール名を取得します。 |

### 3.6. エクスポート操作 & 進捗状況 (`export`)

| 関数 | 戻り値 | 説明 |
| --- | --- | --- |
| `export.get_audio_progress()` | `number` | 音声ミキシングの進捗率（ステレオ・キーフレーム処理、0.0〜1.0）を取得します。 |
| `export.get_progress()` | `number` | 全体のエクスポート進捗率（0.0〜1.0）を取得します。 |

| 関数 | 戻り値 | 説明 |
| --- | --- | --- |
| `export.is_exporting()` | `boolean` | エクスポート処理が実行中かを取得します。 |
| `export.get_status()` | `string` | パイプラインの現在の状態文字列を取得します。 |